11章

設計を進化させる

目まぐるしく変化する今の世の中で、企業はじっとしている余裕はありません。他社との競争で生き残るために、変化を続け、発展し、さらには、自社のやり方を抜本的に変えていくことが必要です。ソフトウェアを設計する時、この現実を無視してはいけません。事業からの要求に適切に対応するために、事業は変化し続けるという現実に、正面から取り組む必要があります。よく考えられたすばらしい設計でも、変化をうまく扱えないソフトウェアは、時間がたつにつれ、修正や拡張がやっかいで危険になります。この章では、ソフトウェア設計で考慮すべき四つの変化のベクトル、つまり、事業活動、組織の構造、業務知識、そしてシステムの成長について検討します。

11.1　事業活動の変化

第2章で、事業を構成する業務領域を次の三つのカテゴリーに分類することを学びました。

中核

競争優位を得るために、競合他社とは異なる独自のやり方で活動する領域

補完

他社とは異なるやり方が必要だが、他社との差別化につながらない領域

一般

どの会社でも同じやり方をする領域

これまでの章で、業務領域のカテゴリーの違いは、設計方針と実装方法の選択に影響することを見てきました。つまり以下の設計判断です。

- 区切られた文脈の境界をどう設計するか
- 文脈どうしをどう連係させるか
- 業務ロジックの複雑さに対応した実装方法のどれを選択するか

事業活動の要求に応えるための設計活動では、対象となる業務領域とそのカテゴリーを特定することがきわめて重要です。それだけではありません。業務領域の進展に機敏に対応することが同じように重要です。企業が成長し進化していくにつれ、ある業務領域のカテゴリーが別のカテゴリーに変化することは、めずらしいことではありません。そういう変化の具体例を見てみましょう。

11.1.1　中核から一般へ

BuyIT社というオンラインの小売事業を考えてみましょう。BuyIT社は、配送コストを他社より低く抑えるために、配送ルートを最適化する革新的なアルゴリズムを開発しました。

ある日、運送業界の今までのやり方を破壊してしまう、DeliveryIT社の新たなサービスが登場しました。この会社は、「巡回セールスマン問題」を解決し、経路最適化を「サービス」として提供する、と主張しています。DeliveryIT社の最適化は、ButIT社が自社で開発したアルゴリズムよりも進化しています。さらにDeliveryIT社のサービスの利用料金は、BuyIT社が自社で同じことを実現する費用の数分の一です。

BuyIT社にとって、DeliveryIT社の経路最適化サービスによって、中核の業務領域が一般的な業務領域に変化しました。結果として、BuyIT社のすべての競合企業は同じレベルの経路最適化機能を使うことが可能になりました。BuyIT社が経路最適化で競争優位性を維持しようとすれば、研究開発に巨額の投資が必要になります。BuyIT社の競争優位性と考えられていた業務領域が、競合他社が購入可能な一般的な業務領域に変化しました。

11.1.2　一般から中核へ

BuyIT社は、創業当初から、在庫管理用のパッケージソフトを使ってきました。しかし、各種の分析レポートは、今の在庫管理のやり方では顧客の需要を適切に予測できていないことを示しています。売れ筋の商品が補充されず、その一方で、売れない商品が在庫スペースを圧迫しています。在庫管理用の他のパッケージソフトを検討した結果、BuyIT社の経営陣は、在庫管理システムの自社開発という戦略的な投資を決定しました。自社独自の在庫管理システムによって、BuyIT社が販売する商品の複雑さをうまく取り扱い、適切な需要予測ができるようになるでしょう。

在庫管理システムをパッケージソフトから自社開発に切り替える決定により、在庫管理は一般的な業務領域から中核の業務領域に変化しました。つまり、在庫管理システムの機能をうまく実現することが、BuyIT社に競争優位をもたらします。競合他社の在庫管理は一般業務のままです。BuyIT社が投資しようとしている先進的な需要予測アルゴリズムを使うことはできません。

一般的な業務領域が中核の業務領域に変化した教科書的な事例がAmazon社です。Amazon社は、他の通信販売事業者と同じように、自社の通信販売事業

を支えるIT基盤が必要でした。Amazon社は、自社のIT基盤を管理する独自の方法を構築し、そのIT基盤を新たな収益源にまで発展させました。つまりAWS（Amazon Web Services）です。

11.1.3　補完から一般へ

BuyIT社の販売促進部門は取引先との契約を管理するシステムを独自に開発していました。この契約管理システムに特別な機能はありません。単純なCRUDアプリケーションです。別の言い方をすれば、補完的な業務領域そのものです。

BuyIT社がこの契約管理システムを自社開発した数年後に、オープンソースの契約管理システムが登場しました。BuyIT社が自社開発したシステムの機能をすべて使えるだけではなく、OCRや全文検索など、もっと進んだ機能も用意されています。これらの機能は、かなり前からBuyIT社の開発バックログに含まれていました。しかし事業への影響は小さいという理由で、優先順位は「低」のままでした。BuyIT社は、自社開発したシステムを見かぎって、このオープンソースの契約管理システムを使うことにしました。契約管理業務のカテゴリーは補完から一般に変化しました。

11.1.4　補完から中核へ

業務領域のカテゴリーが、補完的な業務領域から中核の業務領域に変化することもあります。たとえば、補完的なロジックを最適化することで、コストの削減や新たな収益を生み出す方法を見つけた場合です。

補完的な業務領域の業務ロジックが複雑になることは、この変化が起きる典型的な兆候です。定義上、補完的な業務領域のロジックは単純です。基本的にはCRUD画面やETL処理です。しかし、時間がたつにつれ業務ロジックが入り組んでくるとしたら、それなりの理由があるはずです。会社の利益に貢献しないのに、ロジックが複雑になるとしたら、それは不必要な業務の複雑さです。一方で、もし業務ロジックが複雑になることで収益が増えているとしたら、それは補完的な業務領域が中核の業務領域に変化する兆候です。

11.1.5　中核から補完へ

中核の業務領域は、時間がたつにつれ、補完的な業務領域に変化することがあります。その業務領域の複雑さに適切な理由がなくなった場合、この変化が起きます。別の言い方をすれば、その複雑さが事業収益に貢献しない場合です。このような場合、他の業務領域を支援する必要最小限のロジックだけを残し、収益を生み出さない複雑さを取り除くことになるでしょう。

11.1.6　一般から補完へ

最後になりますが、同じ理由によって一般の業務領域が補完的な業務領域に変化することもあります。さきほどのBuyIT社のオープンソースを利用した契

約管理システムで考えてみましょう。オープンソースの汎用的な仕様に合わせるために、他の社内システムとの連係がかなり複雑になったとします。その複雑さに見合うメリットがないと判断すれば、自社開発に戻すことになるでしょう。つまり、一般的な業務領域が補完的な業務領域に変化します。

ここまで検討した業務領域のカテゴリーの変化をまとめると**図11‐1**になります。

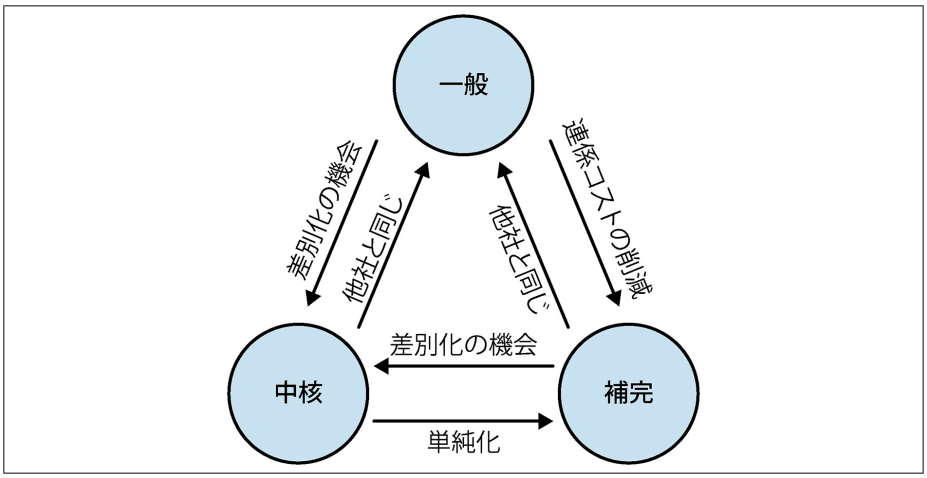

図11-1　業務領域のカテゴリーを変更する理由

11.2　設計の基本方針を再検討する

業務領域のカテゴリーの変化は、区切られた文脈に影響し、その結果、設計方針にも影響します。**第4章**で学んだように、区切られた文脈どうしを連係する方法は業務領域のカテゴリーによって変わります。中核の業務領域ではモデル変換装置を使って自身のモデルを守る必要があります。あるいは、自身のモデルを繰り返し変更しても、連係している他の区切られた文脈に影響を与えないように、公開された言葉（共用サービス）を使う必要があります。

業務領域のカテゴリーが変われば、互いに独立の関係も見直しが必要です。本書の前のほうで説明したように、互いに独立の関係を選択できるのは、補完的な業務領域か一般の業務領域だけです。業務領域のカテゴリーが中核に変わった場合、互いに独立の関係では許容していた機能の重複を放置できなくなります。必然的に、各チームはお互いの実装を統合するしかありません。互いに独立の関係を、利用者と供給者の関係に変えることがもっとも合理的でしょう。中核の業務領域はどちらか一つのチームが開発すべきだからです。

開発戦略にも影響があります。中核の業務領域と補完的な業務領域では、開発のやり方が異なります。補完的な業務領域の開発は外部に委託できるでしょう。あるいは新人の育成機会として使えます。中核の業務領域の開発は、業務知識の提供元との、可能な限りの、緊密な協力が必要です。したがって、補完的な業務領域が中核の業務領域に変化した場合、開発は社内で行うことが必要です。この考え方は、他の変化のパターンにも適用できます。中核の業務領域が補完的な業務領域に変化した場合、その開発を外部に委託することが可能で

す。そうすることで、社内の研究開発チームは、中核の業務領域の開発に専念できます。

11.3　業務ロジックの実装方法を見直す

業務領域のカテゴリーが変化したことを検知する主な指標は、既存の設計では目の前の事業ニーズにうまく対応できない状況です。

さきほど説明した、業務領域のカテゴリーが補完から中核に変化する例で考えてみましょう。補完的な業務領域の業務ロジックの実装は比較的シンプルです。つまり、トランザクションスクリプトかアクティブレコードのいずれかです。**第5章**で説明したように、この二つの実装方法は、複雑なルールと不変条件を含む業務ロジックの実装には向いていません。

入り組んだ業務ルールと不変条件が業務ロジックに追加されていくにつれ、ソースコードはどんどん複雑になります。やがて新しい機能の追加が難しくなります。トランザクションスクリプトやアクティブレコードは、そういう水準の複雑さには対応できません。変更がやっかいで危険であることは重要なシグナルです。変更が難しくなったら事業活動をもう一度分析し、実装方法の選択が適切かどうかを評価し直してください。

実装方法の変更を恐れてはいけません。あたりまえのことです。事業がどのように発展するかは誰も予測できません。また、業務領域のどのカテゴリーに対しても同じように手の込んだ設計をすることは不可能です。時間を無駄に使うだけで、効果的ではありません。状況に応じた適切な設計をして、必要になった時に設計を進化させるようにすべきです。

業務ロジックの実装方法として四つの選択肢があることを知り、それぞれの違いを理解して選択していれば、ある実装方法から別の実装方法への移行は、それほど困難ではありません。業務ロジックの実装方式を変更する例をいくつか紹介します。

11.3.1　トランザクションスクリプトからアクティブレコードへ

トランザクションスクリプトとアクティブレコードは、基本的には、同じ考え方にもとづいています。つまり、どちらも業務ロジックを手続き的なスクリプトとして記述します。両者の違いはデータ構造の扱い方です。アクティブレコードはデータ構造を永続化するための複雑な仕組みをカプセル化します。

結果として、トランザクションスクリプトで扱うデータが込み入ってきた場合、アクティブレコードにリファクタリングします。データ構造が込み入った部分を見つけ、その部分のデータ操作をアクティブレコードオブジェクトにカプセル化します。データベースを直接操作するかわりに、アクティブレコードを使ってモデルと構造を抽象化します。

11.3.2　アクティブレコードからドメインモデルへ

アクティブレコードを操作する業務ロジックが複雑になり、データの不整合やロジックの重複が増えていることに気がついたら、業務ロジックの実装をドメインモデルにリファクタリングしてください。

値オブジェクトを見つけるところから始めます。不変（イミュータブル）なオブジェクトとして表現できそうなデータ構造を特定します。そしてそのデータ構造に関係した業務ロジックを見つけ、値オブジェクトのメソッドとして記述します。

次に、データ構造を分析し、トランザクションの境界を探します。状態を変更する業務ロジックを明確にするために、アクティブレコードのすべてのsetterメソッドをプライベートに変更します。そうすることでアクティブレコードの外部から状態を変更できなくなります。もちろんコンパイルエラーになります。しかしコンパイルエラーのメッセージから、どこで状態を変更しているかを特定できます。状態を変更している操作をアクティブレコードの内側に移動するリファクタリングをします。たとえば次のようなコードです。

```
public class Player
{
  public Guid Id { get; set; }
  public int Points { get; set; }
}

public class ApplyBonus
{
  ...

  public void Execute(Guid playerId, byte percentage)
  {
    var player = _repository.Load(playerId);
    player.Points *= 1 + percentage/100.0;
    _repository.Save(player);
  }
}
```

リファクタリングの最初のステップでは、コードは以下のようになります。このコードはコンパイルエラーになりますが、外部のどのコンポーネントがこのオブジェクトの状態を変える操作をしているか、エラー箇所として特定できます。

```
public class Player
{
  public Guid Id { get; private set; }
  public int Points { get; private set; }
}

public class ApplyBonus
```

```
{
  ...

  public void Execute(Guid playerId, byte percentage)
  {
    var player = _repository.Load(playerId);
    player.Points *= 1 + percentage/100.0;
    _repository.Save(player);
  }
}
```

次のステップで、状態を変更するロジックを、以下のように、アクティブレコードの内部に移動します。

```
public class Player
{
  public Guid Id { get; private set; }
  public int Points { get; private set; }

  public void ApplyBonus(int percentage)
  {
    this.Points *= 1 + percentage/100.0;
  }
}
```

状態を変更するすべてのロジックを適切なオブジェクトの内側に移動できたら、業務ルールと不変条件の一貫性を厳密に保証するためには、どのような階層構造が必要かを検討します。この階層構造が集約の有力な候補です。**第6章**で検討した集約の設計原則を意識して、最小のトランザクション境界を見つけます。別の言い方をすると、一貫性を保証するために必要なデータの、最小の集まりを特定します。オブジェクトの階層構造をそういう最小のトランザクション境界に分解してください。外部の集約を参照する方法はIDだけにします。

最後に、集約ごとにルートとなるオブジェクトを特定します。別の言い方をすると、外部に公開するインターフェースのエントリーポイントを特定します。それ以外の集約内部のメソッドはプライベートとし、外部から呼び出せないようにします。

11.3.3　ドメインモデルからイベント履歴式ドメインモデルへ

集約の境界を適切に設計したドメインモデルは、イベント履歴式ドメインモデルに変換可能です。集約のデータを直接変更するかわりに、集約のライフサイクルを表現する、一連の業務イベントを特定します。

ドメインモデルをイベント履歴式ドメインモデルにリファクタリングする場合、もっともやっかいなのは、すでに存在している集約の履歴をどうするかです。つまり、履歴が失われ、最新状態だけを表現する集約を、イベント履歴式

のモデルに転換する方法です。集約の状態変更の履歴は残っていません。ですから、過去の業務イベントを可能な範囲で修復する、あるいは、移行したことを表現する特別なイベントを用意する、のいずれかの対応が必要です。

11.3.4　過去の状態変化を修復する

このやり方では、移行の対象となる集約が表現していた状態を投影するために、「近似的」なイベント履歴の生成が必要です。**第7章**で使った例で考えてみましょう（**表11‐1**）。

表11-1　最新状態を表現した集約のデータ

lead-id	last-name	first-name	phone-number	status	last-contacted-on	order-placed-on	converted-on
12	小林	俊介	555-4753	成約済	2020-05-27T12:02:12.51Z	2020-05-27T12:02:12.51Z	2020-05-27T12:38:44.12Z

業務ロジックの観点からは、次のような状態の変化があったと想定できます。

1. 見込み客集約を新規に作成した
2. 見込み客に電話をかけた
3. 注文が発行された
4. 支払が終わった（「成約済」になった）

この想定シナリオを業務イベントとして表現すると以下のようになります。

```
{
  "lead-id": 12,
  "event-id": 0,
  "event-type": "新規作成",
  "last-name": "小林",
  "first-name": "俊介",
  "phone-number": "555-4753"
},
{
  "lead-id": 12,
  "event-id": 1,
  "event-type": "連絡済",
  "timestamp": "2020-05-27T12:02:12.51Z"
```

```
  },
  {
    "lead-id": 12,
    "event-id": 2,
    "event-type": "注文発行済",
    "payment-deadline": "2020-05-30T12:02:12.51Z",
    "timestamp": "2020-05-27T12:02:12.51Z"
  },
  {
    "lead-id": 12,
    "event-id": 3,
    "event-type": "支払完了",
    "status": "成約済",
    "timestamp": "2020-05-27T12:38:44.12Z"
  }
```

一連の業務イベントを順番に適用していけば、既存の集約の最新状態を投影できます。この「修復」したイベント履歴が正しいかどうかは、履歴から最新状態を投影した結果と元のデータを比較することで、簡単に確認できます。

しかし、このやり方には欠点があります。イベントソーシングを使う目的は、すべての業務イベントの、信頼できる一貫した履歴を手に入れることです。しかし、この近似的なやり方では、状態変化の履歴の完全な修復はできません。さきほどの例でいえば、販売担当者が見込み客に連絡をとった回数はわかりません。つまり、`架電`イベントの記録がどのくらい失われたかはわかりません。

11.3.5　移行イベントとして表現する

別のやり方として、過去の履歴を失っていることを認めて、「移行」という事実の発生をイベントとして記録する方法があります。現在の状態に至った経緯を推測するのではなく、以下のような「移行イベント」を定義し、それを既存の集約の最初のイベント履歴として使います。

```
  {
    "lead-id": 12,
    "event-id": 0,
    "event-type": "移行",
    "last-name": "小林",
    "first-name": "俊介",
    "phone-number": "555-4753",
    "status": "成約済",
    "last-contacted-on": "2020-05-27T12:02:12.51Z",
    "order-placed-on": "2020-05-27T12:02:12.51Z",
    "converted-on": "2020-05-27T12:38:44.12Z",
    "followup-on": null
  }
```

この方法の利点は、過去の記録が失われたことの明示です。集約の変更履歴が、発生時点から最新状態まで、完全に残っていると誤解してしまうことを防ぎます。欠点は、旧システムの存在がイベント履歴に残り続けることです。たとえば、イベント履歴式ドメインモデルと組み合わせることが多いCQRSを取り入れた場合、最新状態を投影する時に、必ず移行イベントの存在を意識することになります。

11.4　組織の変更

組織の変更はソフトウェアシステムの設計に大きく影響します。第4章で区切られた文脈どうしのさまざまな連係方法を説明しました。つまり、良きパートナー、モデルの共有、従属、モデル変換装置、共用サービス、そして互いに独立です。組織が変われば、開発チーム間の意図の伝達と協力のやり方が変わります。その結果、区切られた文脈の連係方法も変わります。

よくある状況は、開発組織の拡大です（図11‐2）。一つの区切られた文脈の開発を担当するのは一つのチームです。チームが増えれば、広い範囲に区切った一つの区切られた文脈を分割して、それぞれのチームがそれぞれの区切られた文脈を開発することになるでしょう。

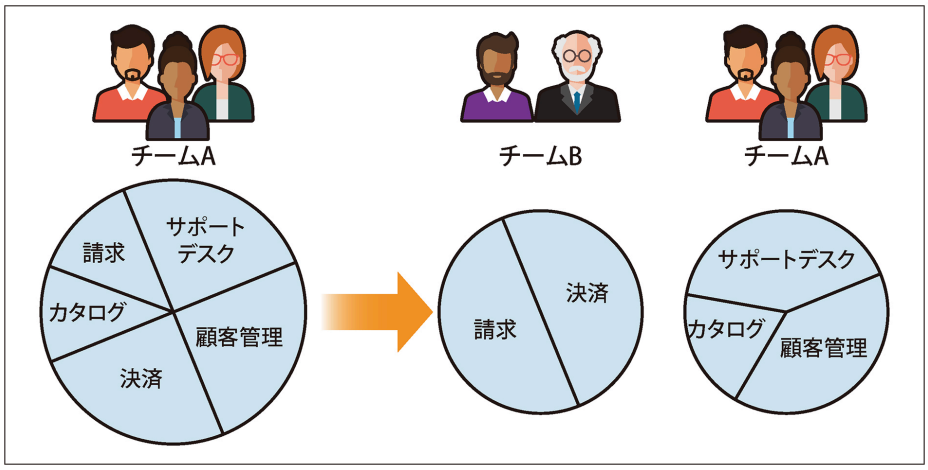

図11-2　チームが増えたら文脈を分けて開発する

それに加え、開発拠点が分散することもあります。ある区切られた文脈の開発を別の拠点でやることになれば、チーム間の協力に悪い影響がでるでしょう。その結果、区切られた文脈の連係方法を進化させることが必要になります。たとえば次のシナリオです。

11.4.1　良きパートナーから利用者と供給者の関係へ

良きパートナーはチーム間の意図の伝達と協力がきわめてうまくいっている場合に成り立ちます。しかし時間がたつにつれて、うまくいかなくなるかもしれません。たとえばどちらかの区切られた文脈を遠く離れた別の拠点のチームが開発することになった場合です。そういう開発体制の変化は、チーム間の意

図の伝達にマイナスに働きます。良きパートナーの関係を、利用者と供給者の関係に分類される三つの方法のどれかに変えることが現実的でしょう。

11.4.2　利用者と供給者の関係から互いに独立へ

残念なことですが、チーム間の意図の伝達がうまくいかないのは、めずらしいことではありません。開発拠点が地理的に離れていることが原因かもしれません。組織の政治的な力学が原因かもしれません。そういう状況では、連係に関連するさまざまな問題が起きがちです。どうしてもうまくいかない場合、お互いが非友好的な無駄なエネルギーを使うより、同じ機能をそれぞれのチームが重複して開発するほうが、費用対効果が高くなることもあるでしょう。

11.5　業務知識

もうおわかりでしょうが、「価値を生み出すソフトウェア設計には業務知識が不可欠である」というのがドメイン駆動設計の中核にある信念です。ソフトウェア開発でもっとも難しい取り組みの一つが、業務知識の習得です。特に、中核の業務領域を理解することは大きな挑戦です。中核の業務領域のロジックは入り組んでいるだけでなく、たびたび変化します。そしてモデル作りがずっと続きます。業務知識が増えるたびにモデルを改善していくことになります。

業務の複雑さは明確ではないことがしょっちゅうです。最初はすべてが単純であたりまえのことに思えます。しかし、多くの場合、それは見せかけの単純さです。あっという間にわけのわからない複雑さに姿を変えてしまいます。機能を追加するたびに例外的な条件分岐が増え、さまざまなルールや不変条件が次々と見つかります。そういう新たな洞察によって、今まで整理してきた秩序が破壊され、モデルを最初から作り直すことになります。区切られた文脈、集約、その他の実装の詳細で定義した、さまざまな境界を変更することになります。

設計方針を決める経験則として、区切られた文脈の境界は、業務知識の深さに合わせるのがよいでしょう。まちがって分割されてしまった区切られた文脈を、後から修正することは、かなり高くつきます。ですから、業務ロジックがよくわかっていない場合、あるいは業務ロジックの変更が多い状況では、広い範囲で文脈を区切るほうが安全です。その後で、開発を進めることで、業務知識が増え、業務ロジックが安定してきたら、広い範囲で区切った文脈を小さな文脈やマイクロサービスに分解することが可能になります。区切られた文脈とマイクロサービスの相互関係は、**第14章**で詳しく検討します。

新たな業務知識を習得したら、その知識を使って設計を進化させ、変更に強い設計にしてください。残念なことに、業務知識の変化はよい方向だけとは限りません。つまり、業務知識が失われることもあります。時間がたつにつれ、文書は更新されなくなり、初期の開発担当者は会社を去り、さまざまな機能がその場しのぎのやり方で追加されます。しまいには、変更がやっかいで危険になり、ソースコードはかなり怪しい状態になります。こういう業務知識の劣化がひどくならないように努力することは、きわめて重要です。失われた業務知

識を取り戻すよい方法として、イベントストーミングがあります。このやり方は次の章で説明します。

11.6　システムの成長

システムの成長は、ソフトウェアが健全である証（あかし）です。機能の追加が続くのは、システムがうまくいっている証拠です。そのシステムは利用者に価値を提供し、さらなる要求に応え、競合プロダクトと対抗できています。しかしソフトウェアの成長は、よいことばかりではありません。開発規模が大きくなるにつれ、そのソフトウェアは大きな泥団子になってしまうかもしれません。

大きな泥団子は、その場しのぎの設計で、だらしなく、いいかげんで、つぎはぎだらけになったスパゲッティコードのジャングルです。無秩序に機能を追加し、手っ取り早いやり方で修正を繰り返してきたことの、何よりの証拠です。

——ブライアン・フートとジョセフ・ヨーダー[＊1]

[＊1]　Brian Foote、Joseph Yoder. Big Ball of Mud. Fourth Conference on Patterns Languages of Programs (PLoP '97/EuroPLoP '97)、モンティチェロ（イリノイ州）、1997年9月.

システムが無秩序に成長し、大きな泥団子になってしまうのは、機能を拡張する時に設計を再検討していないからです。成長することで、コンポーネントの境界が膨張し、ますます機能がそこに追加されます。ドメイン駆動設計の主要な関心事は「境界」をどう設定するかです。つまり、事業活動を構成する要素（業務領域）、モデル（区切られた文脈）、不変性（値オブジェクト）、そして一貫性（集約）のそれぞれの境界の設定です。したがって、ソフトウェアの成長が、設計判断、つまり境界の設定にどう影響するかの分析は、きわめて重要です。

成長に起因する複雑さに取り組むよい方法は、必要のない複雑さ、つまり過去の設計判断に起因する複雑さを特定し取り除くことです。本質的な複雑さ、あるいは事業活動が持つ本来の複雑さに対しては、ドメイン駆動設計の手法を使って対処します。

今まで検討してきたように、ドメイン駆動設計の基本的な進め方は次のとおりです。

- 事業活動と業務領域を分析する
- 対象とする業務領域のモデルを作る
- モデルにもとづいてソフトウェアを設計し実装する

このやり方で、成長によって増大する複雑さに取り組んでみましょう。

11.6.1　業務領域

第1章で検討したように、業務領域の境界を特定することは、簡単ではありません。ですから、完璧な境界を見つけようとがんばらず、役に立つ境界を探すことに全力をつくすべきです。どういうことかというと、業務領域の特定は、さまざまな事業活動がどのような事業価値を提供しているかを識別し、ソフトウェアの実装方法を適切に選択することにつながります。

事業が成長すれば、業務領域の境界がぼやけてくることがあります。 ある業務領域が、実際はもっと小さな複数の業務領域にまたがっている活動であることに気がつきにくくなります。ですから、業務領域の分け方を見直すことがたいせつです。強く関係するユースケース、つまり同じデータ集合を操作するユースケースの集合に注目する、という経験則を使って、業務領域の適切な切れ目を見つけてください（図11‐3）。

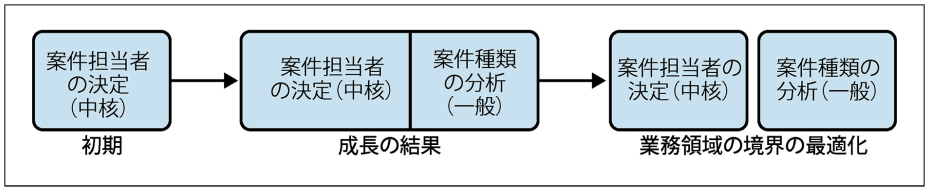

図11-3　成長に応じた業務領域の最適化

業務領域を小さく分け、そのカテゴリーを特定できれば、事業活動の本質的な複雑さを扱うための、重要な知見が手に入ります。業務領域について正確な情報を入手できれば、それぞれの業務領域に適した実装方法を効果的に判断できます。

大きな業務領域から小さな業務領域を特定することは、中核の業務領域では特に大きな意味を持ちます。純度の高い中核の業務領域を抽出することを常に目指してください。さまざまな業務領域の中から中核の業務領域を抜き出すことで、事業戦略上もっとも重要な領域に開発資源を投入できます。

11.6.2　区切られた文脈

第3章で学んだように、区切られた文脈という道具を使えば、さまざまな文脈で事業活動を解釈し、別々のモデルを作ることが可能です。「何にでも手を出して、どれもちゃんとできていない」モデルを作ってはいけません。さまざまな課題をうまく解決するためには、課題に合わせた複数のモデルを作ってください。

開発の規模が大きくなるにつれ、区切られた文脈の焦点がぼやけてきます。異なる課題を解決するためのロジックがごちゃまぜになってきます。これは不必要な複雑さです。業務領域の特定と同じように、区切られた文脈の境界を時々見直すことがきわめて重要です。モデルを単純に保つことを常に心がけて

ください。特定課題の解決に焦点がぴたりと合った区切られた文脈を抽出してください。

システムの成長は、設計の潜在的な問題を明らかにします。たとえば、区切られた文脈どうしでかなりの通信が発生していることに気がつくかもしれません。あらゆる操作の実行で、他の区切られた文脈と通信をしています。モデルが適切でないことは明らかです。区切られた文脈の自律性を高めるために、文脈の区切り方を見直さなければいけません。

11.6.3　集約

第6章では、業務ロジックの実装方法としてドメインモデルを検討しました。その中で集約の境界を設計する指針として以下を説明しました。

経験則として、集約はできるだけ小さく設計すべきです。一貫性を保証する対象は、集約に記述する業務ロジックが必要とするデータだけに限定します。

システムの業務要件が増えてくると、集約はできるだけ小さく設計するという指針を忘れ、既存の集約のあちこちにコードを追加して機能を実現したほうが手っ取り早い、と感じるかもしれません。しかし、業務ロジックとして守るべき一貫性とは無関係なデータを集約に追加すると、必要のない複雑さを集約に持ち込んでしまいます。そういう不必要な複雑さは取り除かなければいけません。

業務機能を専用の集約に抽出することで、元の集約が単純化されるだけでなく、その集約が所属する区切られた文脈が単純化される可能性があります。集約を単純化するリファクタリングによって、新たな区切られた文脈として抽出すべき隠れていた別のモデルを発見することは、少なくありません。

11.7　まとめ

人生において唯一不変なことは変化である。

ギリシアの哲学者ヘラクレイトスの言葉です。事業活動も同じです。競争上の優位を維持するには、事業活動を常に進化させ変えていく必要があります。事業活動のこういう変化は、ソフトウェア設計で考慮すべき、きわめて重要な要素として取り扱う必要があります。

事業活動の発展に伴う業務領域のさまざまな変化を識別し、それを設計に反映してください。過去の設計判断が、現在の事業方針や業務活動と整合しているかどうかを確認してください。必要であれば、現在の事業方針と一致するように設計を進化させてください。

組織の変更は、チーム間の意図の伝達と協力関係に大きく影響します。そして区切られた文脈どうしの連係方法に影響します。また事業活動を学ぶ活動に

終わりはありません。時間とともに業務知識を増やし、その知識を使って設計方針を検討し、実装レベルの設計判断に活かしてください。

最後に、ソフトウェアの成長は望ましい変化です。しかし、その変化をうまく扱わないと、ソフトウェアの設計やシステムの構造に、やっかいな問題を持ち込みます。ですから、以下を意識することがたいせつです。

- 業務領域の機能を拡張することが必要になったら、適切な設計判断をするために、業務領域を小さい単位で識別することを検討してみましょう。
- 「何にでも手を出して、どれもちゃんとできていない」区切られた文脈を作らないでください。区切られた文脈で表現するモデルは、特定の課題を解決することに集中してください。
- 集約はできるだけ小さく設計してください。強い一貫性が必要なデータの範囲を特定する、という経験則を使って、業務ロジックを新しい集約に抽出する可能性を探ってください。

この章の最後のアドバイスです。成長に起因する複雑さの増大を検知するために、さまざまな境界を常に監視してください。必要のない複雑さを取り除きましょう。そして、ドメイン駆動設計の考え方とやり方を活用して、事業活動の本質的な複雑さに取り組みましょう。

11.8　演習問題

1. 組織が拡大すると区切られた文脈の連係方法はどう変わりそうですか？
   A. 良きパートナーから利用者と供給者の関係（従属、モデル変換装置または共用サービス）
   B. モデル変換装置から共用サービス
   C. 従属からモデルの共有
   D. 共用サービスからモデルの共有
2. 区切られた文脈の連係方法が、従属から互いに独立に変化するとしたら、どういう状況が考えられますか？
   A. 他のチームと協力することに苦労している
   B. 重複している機能は補完的な業務領域か一般的な業務領域の機能である
   C. 重複している機能は中核の業務領域の機能である
   D. AとB
   E. AとC
3. 補完的な業務領域が中核の業務領域に変わる兆候はどれですか？
   A. 既存のモデルの改善と機能の追加が楽になってきた
   B. 既存のモデルの改善がたいへんになってきた
   C. 変更の頻度が多い
   D. BとC
   E. どれもあてはまらない
4. 新たな事業機会を見つけた時に起きる変化はどれですか？
   A. 補完的な業務領域が中核の業務領域に変わる
   B. 補完的な業務領域が一般的な業務領域に変わる

C. 一般的な業務領域が中核の業務領域に変わる

D. 一般的な業務領域が補完的な業務領域に変わる

E. AとB

F. AとC

5. ウルフデスク社（「この本について」で説明した架空の事業会社）で、一般的な業務領域を中核の業務領域に変更するとしたら、事業戦略をどう変えた時でしょう？